Programmation statistique automatisée

BUT Science des Données - Metz

Berhocoïrigoin Elena

Avant-propos

- N'hésitez pas à poser des questions avant, pendant, après

Objectifs

Python pour …

- Lecture de données
- Traitement de données
- Extraction d'informations (via statistiques, graphiques)
- Automatisation des traitements

Pandas

Pandas

Pandas est une librairie python

Module : **Fichier Python** contenant un **ensemble de fonctions** et de **variables globales**. Extension `.py`.

Package : **Répertoire** contenant un **ensemble de modules** et un fichier `__init__.py`, grâce auquel l'interpréteur interprète le répertoire comme un package. Un package peut également contenir des sous-packages.

Librairie : Définition moins claire. Sens généralement admis : ensemble de packages.

Pandas

- **Pandas** est une librairie Python construite à partir de Numpy, qui simplifie le travail de traitement et d'analyse des données

Pandas

- **Pandas** est une librairie Python construite à partir de Numpy, qui simplifie le travail de traitement et d'analyse des données
- L'objet principal en Pandas est le **DataFrame**, un tableau à deux dimensions : lignes et colonnes

Pandas

- **Pandas** est une librairie Python construite à partir de Numpy, qui simplifie le travail de traitement et d'analyse des données
- L'objet principal en Pandas est le **DataFrame**, un tableau à deux dimensions : lignes et colonnes
  - Les **colonnes** peuvent être de **types différents**, contrairement à Numpy
  - Un DataFrame contient des **valeurs**, accessible via la méthode `values`
  - Un DataFrame possède des **labels de lignes et de colonnes,** accessibles via les méthode `index` et `columns`, contrairement aux tableaux Numpy qui ne possèdent que des index implicites

indices de position

implicite

0 1 2 3

0

1

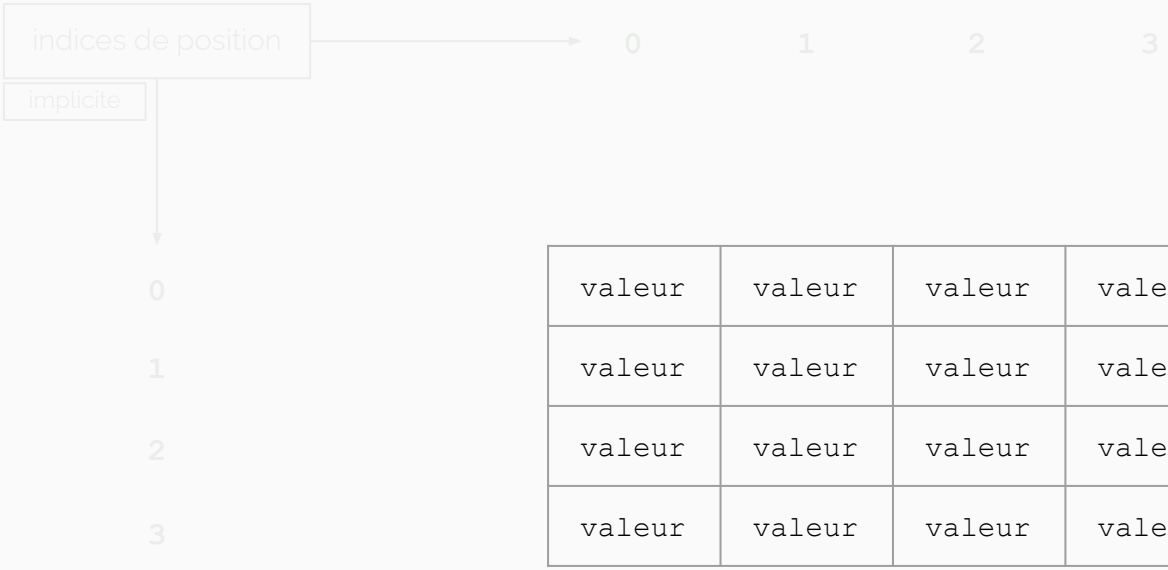
2

3

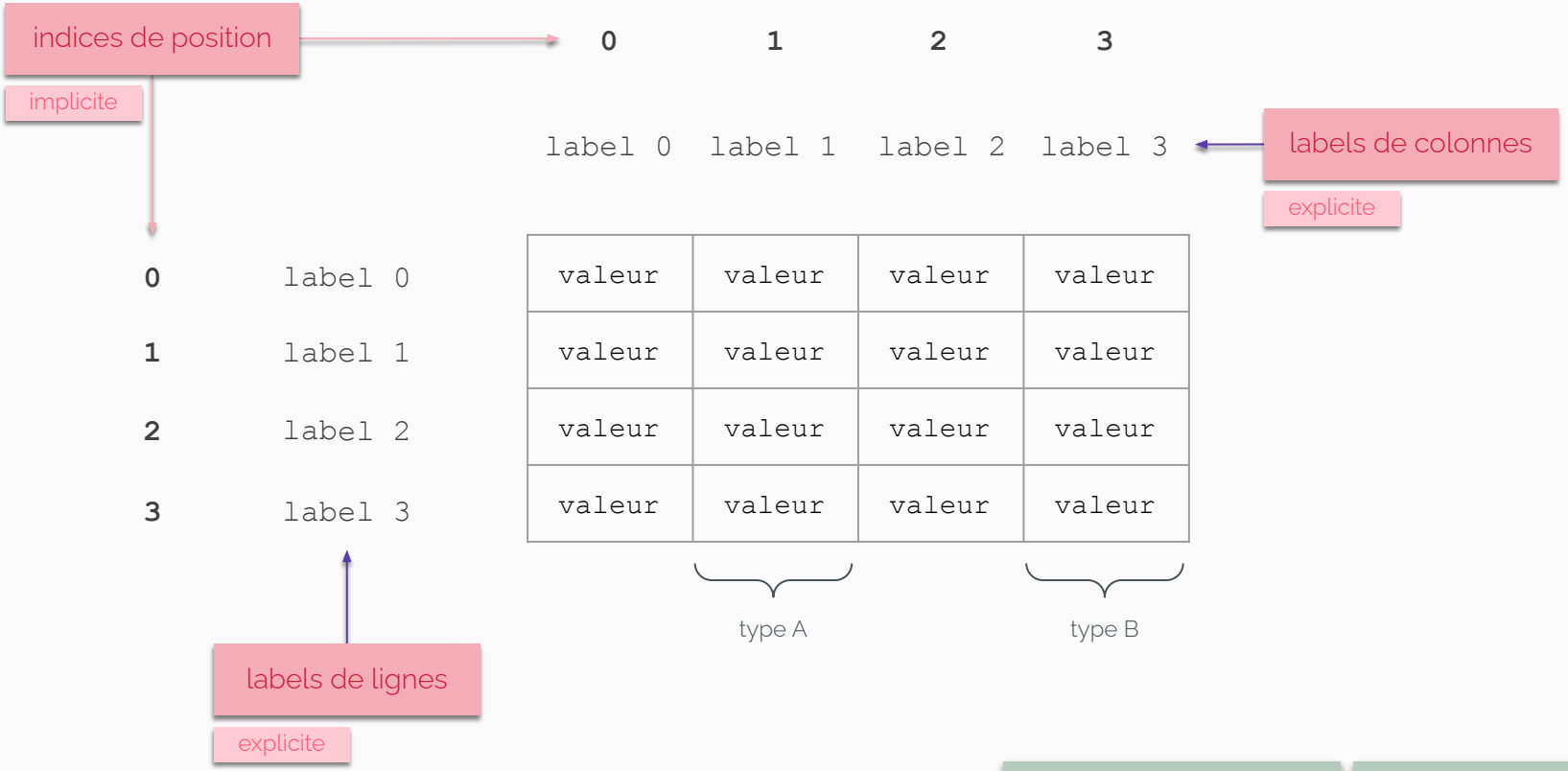
valeur	valeur	valeur	valeur
valeur	valeur	valeur	valeur
valeur	valeur	valeur	valeur
valeur	valeur	valeur	valeur

type A

Array Numpy


indices de position
implicite
0
1
2
3
0
1
2
3
valeur valeur valeur valeur
valeur valeur valeur valeur
valeur valeur valeur valeur
valeur valeur valeur valeur

```
print(arr)
```



DataFrame Pandas

indices de position

implicite

0 1 2 3

label 0 label 1 label 2 label 3

labels de colonnes

explicite

0	label 0	valeur	valeur	valeur	valeur
1	label 1	valeur	valeur	valeur	valeur
2	label 2	valeur	valeur	valeur	valeur
3	label 3	valeur	valeur	valeur	valeur

labels de lignes

explicite

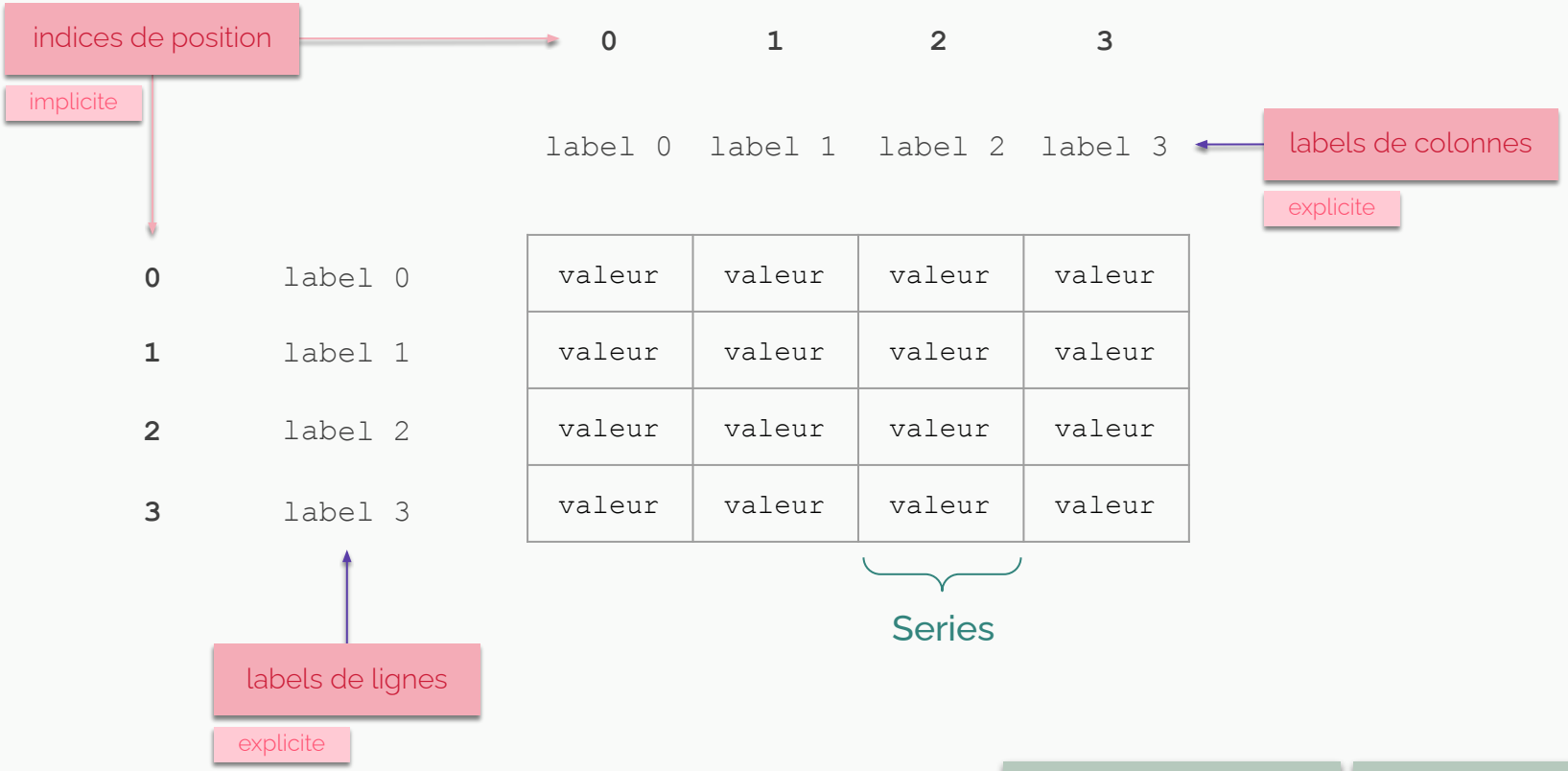
```
print(df)
```

Pandas

- **Pandas** est une librairie Python construite à partir de Numpy, qui simplifie le travail de traitement et d'analyse des données
- L'objet principal en Pandas est le **DataFrame**, un tableau à deux dimensions : lignes et colonnes
  - Les **colonnes** peuvent être de **types différents**, contrairement à Numpy
  - Un DataFrame contient des **valeurs**, accessible via la méthode `values`
  - Un DataFrame possède des **labels de lignes et de colonnes** accessibles via la méthode `index`, contrairement aux tableaux Numpy qui ne possèdent que des index implicites

Pandas

- **Pandas** est une librairie Python construite à partir de Numpy, qui simplifie le travail de traitement et d'analyse des données
- L'objet principal en Pandas est le **DataFrame**, un tableau à deux dimensions : lignes et colonnes
  - Les **colonnes** peuvent être de **types différents**, contrairement à Numpy
  - Un DataFrame contient des **valeurs**, accessible via la méthode `values`
  - Un DataFrame possède des **labels de lignes et de colonnes** accessibles via la méthode `index`, contrairement aux tableaux Numpy qui ne possèdent que des index implicites
  - Un DataFrame est une collection d'objets d'une dimension appelés **Series**
- Les **Series** sont des extensions des arrays unidimensionnels Numpy (notamment : types de variables supplémentaires)



DataFrame Pandas

Pandas

- **Créer** un DataFrame à partir de :
  - liste ou liste de listes via `pd.DataFrame()`
  - fichier CSV et dérivés, tableurs, fichier Parquet, fichier JSON … via `pd.read_csv()`

Pandas

- **Créer** un DataFrame à partir de :
  - liste ou liste de listes via `pd.DataFrame()`
  - fichier CSV et dérivés, tableurs, fichier Parquet, fichier JSON … via `pd.read_csv()`

Remarque :

Dans un ordinateur vous avez :

- **mémoire de masse** : **stockage à long terme** même en l'absence d'alimentation électrique
  - **lenteur d'accès** (accès à une donnée = ms)

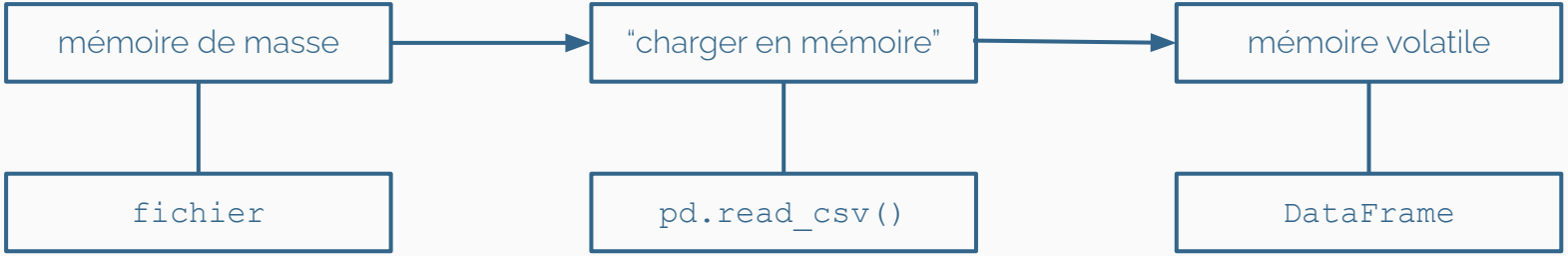
  exemple : disque dur, SSD
- **mémoire volatile** ou **mémoire vive** : **stockage transitoire**, perte de toutes les données en mémoire dès qu'elle cesse d'être alimentée en électricité
  - **rapidité d'accès** (accès à une donnée = ns) : 1 million de fois plus rapide

  exemple : RAM

Pandas

- **Créer** un DataFrame à partir de :
  - liste ou liste de listes via `pd.DataFrame()`
  - fichier CSV et dérivés, tableurs, fichier Parquet, fichier JSON … via `pd.read_csv()`

Remarque :



Pandas

- **Créer** un DataFrame à partir de :
  - liste ou liste de listes via `pd.DataFrame()`
  - fichier CSV et dérivés, tableurs, fichier Parquet, fichier JSON … via `pd.read_csv()`
- **Accéder** à un ou plusieurs éléments :
  - via les **indices implicites**, les **indices de position** (“numéro de la ligne, numéro de la colonne”) :

    ```
    <DataFrame>.iloc[<lignes_pos>, <colonnes_pos>]
    ```

Pandas

- **Créer** un DataFrame à partir de :
  - liste ou liste de listes via `pd.DataFrame()`
  - fichier CSV et dérivés, tableurs, fichier Parquet, fichier JSON … via `pd.read_csv()`
- **Accéder** à un ou plusieurs éléments :
  - via les **indices implicites**, les **indices de position** (“numéro de la ligne, numéro de la colonne”) :

    ```
    <DataFrame>.iloc[<lignes_pos>, <colonnes_pos>]
    ```

  - via les **indices explicites**, les **labels des lignes et des colonnes** (“nom de la ligne, nom de la colonne”) :

    ```
    <DataFrame>.loc[<lignes_lab>, <colonnes_lab>]
    ```

Pandas

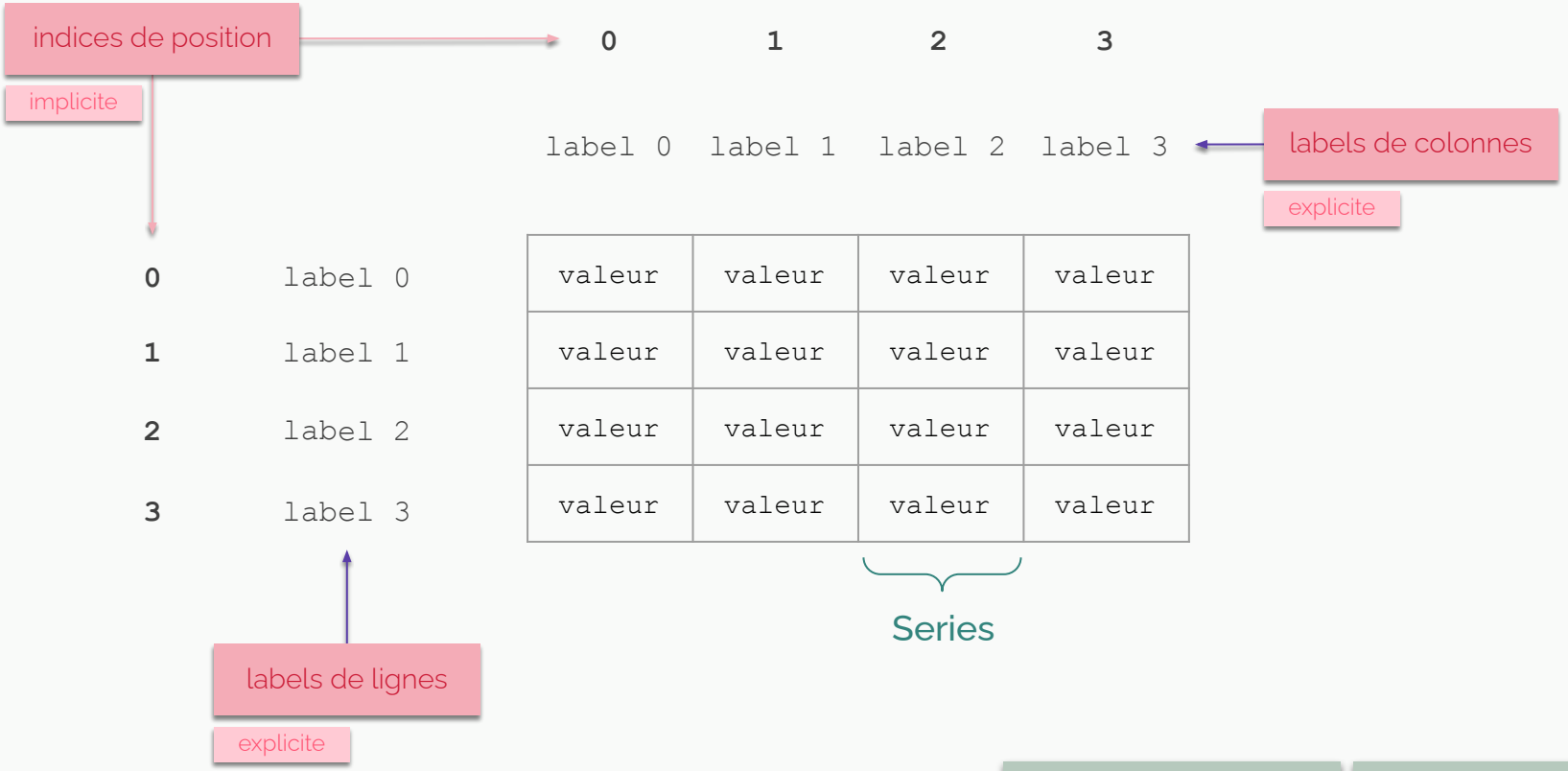
- **Créer** un DataFrame à partir de :
  - liste ou liste de listes via `pd.DataFrame()`
  - fichier CSV et dérivés, tableurs, fichier Parquet, fichier JSON … via `pd.read_csv()`
- **Accéder** à un ou plusieurs éléments :
  - via les **indices implicites**, les **indices de position** :

    ```
    <DataFrame>.iloc[<lignes_pos>, <colonnes_pos>]
    ```

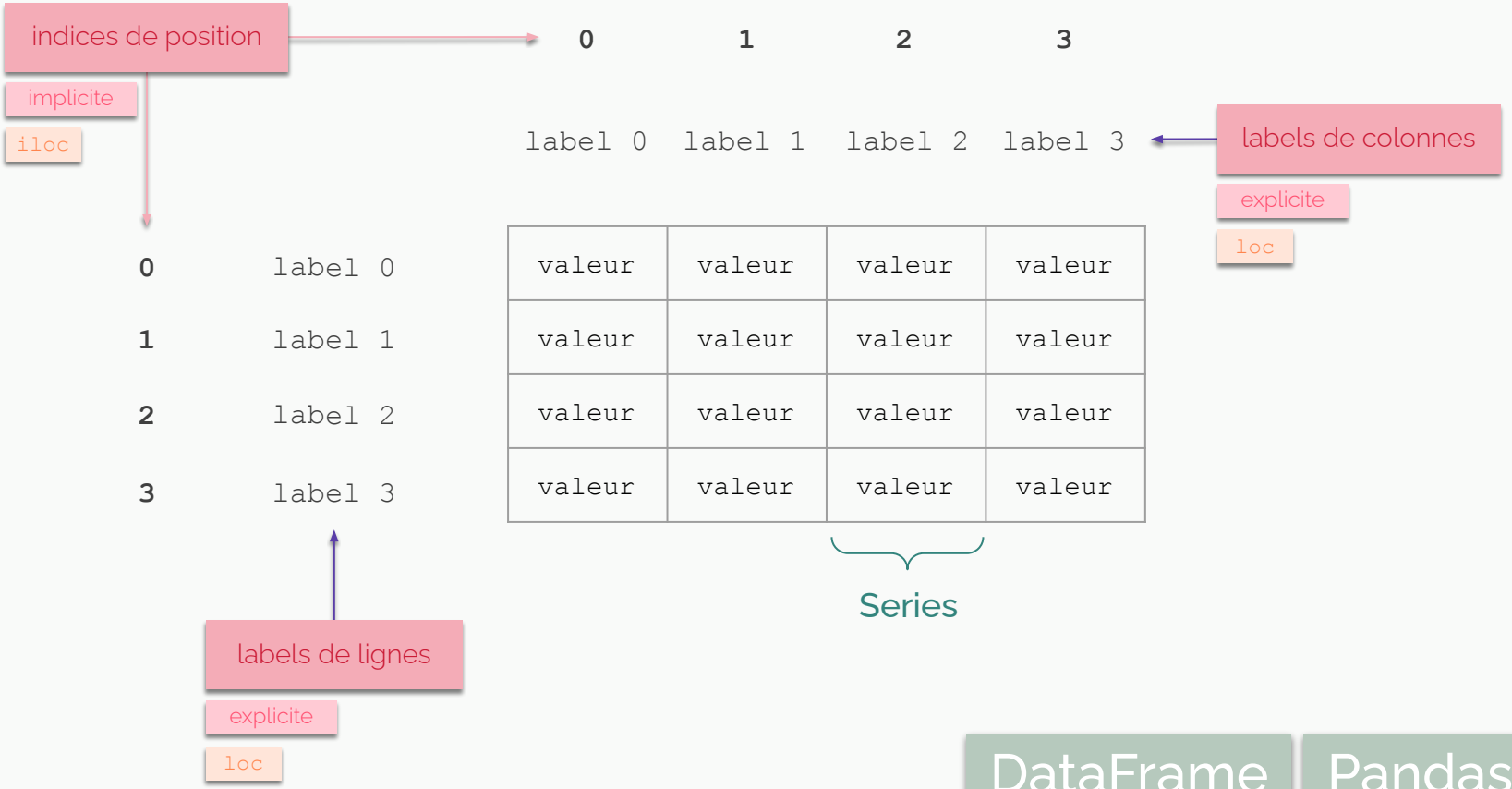
  - via les **indices explicites**, les **labels des lignes et des colonnes** :

    ```
    <DataFrame>.loc[<lignes_lab>, <colonnes_lab>]
    ```

  - via :
    - Indexation classique
    - Slicing
    - Filtre logique



DataFrame Pandas


indices de position
implicite
iloc
0
1
2
3
label 0
label 1
label 2
label 3
labels de colonnes
explicite
loc
0
1
2
3
label 0
label 1
label 2
label 3
valeur
Series
labels de lignes
explicite
loc
DataFrame
Pandas

indices de position

implicite

iloc

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

		0	1	2	3
		jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

indices de position

implicite

iloc

labels de colonnes

explicite

loc

		0	1	2	3
		jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

labels de lignes

explicite

loc

indices de position

implicite

`iloc`

0	1	2	3

labels de colonnes

explicite

`loc`

	jour	mois	annee	age
0 Tom	1	12	2003	21
1 Paule	4	5	1996	29
2 Sophie	7	8	1967	58
3 Eden	10	11	2012	12

labels de lignes

explicite

`loc`

indices de position

implicite

`iloc`

0	1	2	3

labels de colonnes

explicite

`loc`

	jour	mois	annee	age
0 Tom	1	12	2003	21
1 Paule	4	5	1996	29
2 Sophie	7	8	1967	58
3 Eden	10	11	2012	12

labels de lignes

explicite

`loc`

indices de position

implicite

iloc

		0	1	2	3
		jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

```
df.iloc[ligne,colonne]
```

indices de position

implicite

iloc

	0	1	2	3
	jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

```
df.iloc[2,1]
```

indices de position

implicite

iloc

	0	1	2	3
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

jour mois annee age

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

indices de position

implicite

iloc

		0	1	2	3
		jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

indices de position

implicite

iloc

		0	1	2	3
		jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

```
df.loc[ligne,colonne]
```

indices de position

implicite

iloc

		0	1	2	3
		jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

```
df.loc["Sophie","mois"]
```

indices de position

implicite

iloc

labels de colonnes

explicite

loc

		0	1	2	3
		jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

labels de lignes

explicite

loc

```
df.iloc[2,1]

df.loc["Sophie","mois"]
```

indices de position

implicite

iloc

labels de colonnes

explicite

loc

		0	1	2	3
		jour	mois	annee	age
0	0	1	12	2003	21
1	1	4	5	1996	29
2	2	7	8	1967	58
3	3	10	11	2012	12

labels de lignes

explicite

loc

```
df.iloc[2,1]

df.loc[2,"mois"]
```

indices de position

implicite

iloc

labels de colonnes

explicite

loc

		0	1	2	3
		0	1	2	3
0	0	1	12	2003	21
1	1	4	5	1996	29
2	2	7	8	1967	58
3	3	10	11	2012	12

labels de lignes

explicite

loc

```
df.iloc[2,1]

df.loc[2,1]
```

indices de position

implicite

`iloc`

0	2
1	1
2	3
3	0

labels de lignes

explicite

`loc`

0	1	2	3
1	12	2003	21
4	5	1996	29
7	8	1967	58
10	11	2012	12

0	2	1	3

labels de colonnes

explicite

`loc`

```
df.iloc[2,1] = ?

df.loc[2,1] = ?
```

indices de position

implicite

`iloc`

labels de colonnes

explicite

`loc`

0		**1**	**2**	**3**	
		0	2	1	3
0	2	1	12	2003	21
1	1	4	5	1996	29
2	3	7	8	1967	58
3	0	10	11	2012	12

labels de lignes

explicite

`loc`

```
df.iloc[2,1] = 8

df.loc[2,1] = ?
```

indices de position

implicite

iloc

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

		0	1	2	3
		0	2	1	3
0	2	1	12	2003	21
1	1	4	5	1996	29
2	3	7	8	1967	58
3	0	10	11	2012	12

```
df.iloc[2,1] = 8

df.loc[2,1] = 2003
```

indices de position

implicite

iloc

labels de colonnes

explicite

loc

labels de lignes

explicite

loc

		0	1	2	3
		0	2	1	3
0	2	1	12	2003	21
1	1	4	5	1996	29
2	3	7	8	1967	58
3	0	10	11	2012	12

```
df.iloc[2,1] = 8

df.loc[2,1] = 2003
```

Pandas

- **Accéder** à un ou plusieurs éléments :
  - via les **indices implicites**, les **indices de position** :

    ```
    <DataFrame>.iloc[<lignes_pos>, <colonnes_pos>]
    ```

  - via les **indices explicites**, les **labels des lignes et des colonnes** :

    ```
    <DataFrame>.loc[<lignes_lab>, <colonnes_lab>]
    ```

  - via :
    - Indexation classique
    - Slicing
    - Filtre logique

Pandas

- **Accéder** à un ou plusieurs éléments :
  - via les **indices implicites**, les **indices de position** :

    `<DataFrame>.iloc[<lignes_pos>, <colonnes_pos>]`
  - via les **indices explicites**, les **labels des lignes et des colonnes** :

    `<DataFrame>.loc[<lignes_lab>, <colonnes_lab>]`
  - via :
    - Indexation classique
    - Slicing
    - Filtre logique
- Cas particuliers :
  - Accéder à une ou plusieurs colonnes :
    - `<DataFrame>[<colonnes_lab>]`
    - `<DataFrame>.<colonne_lab>` (une seule colonne)
  - Filtre logique sur les lignes :
    - `<DataFrame>[<expression>]`

Pandas

- **Modifier** : sélection et affectation, fonctions numpy ou pandas, combinaison de plusieurs DataFrame

Pandas

Rappel :

On peut modifier une variable en écrasant l'ancienne valeur par la nouvelle valeur via l'affectation

<u>Exemple</u> :

```
x = 2
print("before :", x)
x = x + 3
print("after :", x)
```

```
before : 2
after : 5
```

```
y = [1, 2]
print("before :", y)
y = y + [3, 4]
print("after :", y)
```

```
before : [1, 2]
after : [1, 2, 3, 4]
```

Pandas

- **Modifier** : sélection et affectation, fonctions numpy ou pandas, combinaison de plusieurs DataFrame
- **Agrégation** : fonction Pandas pour statistiques descriptives simples

		0	1	2	3	4
		nom	jour	mois	annee	age
0	0	Tom	1	12	2003	21
1	1	Paule	4	5	1996	29
2	2	Sophie	7	8	1967	58
3	3	Eden	10	11	2012	12

```
df = df.set_index("nom")
```

		0	1	2	3
	nom	jour	mois	annee	age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

		0 nom	1 jour	2 mois	3 annee	4 age
0	0	Tom	1	12	2003	21
1	1	Paule	4	5	1996	29
2	2	Sophie	7	8	1967	58
3	3	Eden	10	11	2012	12

```
df = df.set_index("nom")
```

	nom	0 jour	1 mois	2 annee	3 age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

		0 nom	1 jour	2 mois	3 annee	4 age
0	0	Tom	1	12	2003	21
1	1	Paule	4	5	1996	29
2	2	Sophie	7	8	1967	58
3	3	Eden	10	11	2012	12

```
df.iloc[0,1] = 1
df.loc[0,"jour"] = 1
```

```
df.iloc[0,1] = 12
df.loc["Tom","jour"] = 1
```

```
df = df.set_index("nom")
```

	nom	0 jour	1 mois	2 annee	3 age
0	Tom	1	12	2003	21
1	Paule	4	5	1996	29
2	Sophie	7	8	1967	58
3	Eden	10	11	2012	12

Résumé des fonctions / méthodes vues

Création :

- **pd.Series()**
- **pd.DataFrame()**
- **pd.read_csv()**

Propriétés :

- <DataFrame>**.values**
- <DataFrame>**.index**
- <DataFrame>**.columns**
- <DataFrame>**.ndim**
- <DataFrame>**.size**
- <DataFrame>**.shape**
- <DataFrame>**.dtypes**
- **len**(<DataFrame>)
- <DataFrame>**.head()**
- <DataFrame>**.tail()**

Résumé des fonctions / méthodes vues

Indexation classique ou Slicing :

- `<DataFrame>.iloc[<lignes_pos>, <colonnes_pos>]`
- `<DataFrame>.loc[<lignes_lab>, <colonnes_lab>]`
- `<DataFrame>[<colonnes_lab>]`
- `<DataFrame>.<colonne_lab>` (une seule colonne)

Filtre logique :

- `<DataFrame>[<expression>]`
- `<DataFrame>.loc[<expression>, <colonnes_lab>]`
- `<DataFrame>.loc[<lignes_lab>, <expression>]`
- `& | ~`
- `<DataFrame>.isna()`
- `<DataFrame>.any()`
- `<DataFrame>.all()`
- `<DataFrame>.isin()`

Résumé des fonctions / méthodes vues

Modification :

- <DataFrame>.**set_index(**<colonne>**)**
- <DataFrame>.**reset_index()**
- <DataFrame>.**copy()**
- **pd.concat(**[...]**)**
- +, x, -, /
- <DataFrame>[...] **=** …
- <DataFrame>.iloc[...,...] **=** ...
- <DataFrame>.loc[...,...] **=** ...
- <DataFrame> **=** ...
- <DataFrame>.**drop()**
- <DataFrame>.**rename()**
- <DataFrame>.**sort_values()**
- <DataFrame>.**sort_index()**
- <DataFrame>.**T**

Statistiques :

- <Series>.**unique()**
- <DataFrame_or_Series> .**count()**
- <DataFrame_or_Series> .**nunique()**
- <DataFrame_or_Series> .**value_counts()**
- <DataFrame_or_Series> .**sum()**
- <DataFrame_or_Series> .**mean()**
- <DataFrame_or_Series> .**min()**
- <Series>.**argmin()**
- <DataFrame_or_Series> .**max()**
- <Series>.**argmax()**
- <DataFrame_or_Series> .**std()**
- <DataFrame_or_Series> .**median()**
- <DataFrame>.**corr()**
- <DataFrame>.**agg()**

Résumé des fonctions / méthodes vues

Statistiques :

- `<Series>.quantile()`

Autre :

- `<DataFrame>.duplicated()`
- `<DataFrame>.drop_duplicates()`

MERCI